

# Lab Worksheets – 2307421L- Ubiquitous Sensing, Computing and Communication (USCC)

## Practical NO. 2: Mobile App Development for Sensor Control

**Batch:** Batch 2

**Group No.:**

**Members:**

- Rushikesh Sable (202201070107)

**Objective:**

- To develop a mobile application that displays real-time agricultural sensor data (temperature, humidity, soil moisture, and pH).
- To integrate Firebase Realtime Database for storing and retrieving data.
- To design a Bolt.AI mobile app for visualization of real-time readings.
- To enable live monitoring of environmental parameters for smart farming.

**Tools & Technology Used:**

| Tool / Component           | Purpose                                                    |
|----------------------------|------------------------------------------------------------|
| ESP8266 (NodeMCU)          | Microcontroller for sensing and Wi-Fi connectivity         |
| DHT11 Sensor               | Measures temperature and humidity                          |
| Soil Moisture Sensor       | Detects soil water level                                   |
| pH Sensor                  | Measures soil acidity/alkalinity                           |
| Firebase Realtime Database | Cloud platform for live data storage                       |
| Bolt.AI App Builder        | Used to develop mobile app UI for displaying live readings |
| Wi-Fi Network              | For connecting ESP8266 to Firebase                         |

## **Procedure:**

### **1. Hardware Setup**

- Connected DHT11, soil moisture, and pH sensors to ESP8266 NodeMCU.
- Verified pin connections according to datasheets.
- Powered ESP8266 via USB and confirmed COM port detection.

### **2. Firebase Configuration**

- Created a new Firebase project and enabled Realtime Database.
- Set read/write rules in test mode (".read": true, ".write": true).
- Copied the Database URL and Authentication Key.

### **3. Code Implementation**

- Programmed ESP8266 using Arduino IDE.
- Installed required libraries: *ESP8266WiFi.h*, *FirebaseESP8266.h*, and *DHT.h*.
- Configured Wi-Fi credentials and Firebase details.
- Uploaded data from DHT11, pH, and moisture sensors to Firebase.

### **4. App Design (Bolt.AI)**

- Created a Bolt.AI project and connected it with Firebase.
- Designed UI blocks showing: Temperature, Humidity, pH Level, and Soil Moisture.
- Added real-time text and chart components for visualization.

### **5. Testing & Validation**

- Verified that data appeared in Firebase Realtime Database.
- Confirmed the same readings appeared instantly on the mobile app.

**Code Snippets / Configuration:-**

```

app > (tabs) > _layout.tsx

1  import { Tabs } from 'expo-router';
2  import { Leaf, Settings } from 'lucide-react-native';
3
4  export default function TabLayout() {
5    return (
6      <Tabs
7        screenOptions={{
8          headerShown: false,
9          tabBarActiveTintColor: '#10b981',
10         tabBarInactiveTintColor: '#9ca3af',
11         tabBarStyle: {
12           borderTopColor: '#e5e7eb',
13           borderTopWidth: 1,
14           paddingBottom: 8,
15           paddingTop: 8,
16           height: 70,
17         },
18         tabBarLabelStyle: {
19           fontSize: 12,
20           marginTop: 4,
21         },
22       }}
23     >
24       <Tabs.Screen
25         name="index"
26         options={{
27           title: 'Monitor',
28           tabBarIcon: ({ size, color }) => <Leaf size={size} color={color} />,

```

Fig:- layout structure

```

config > firebase.ts

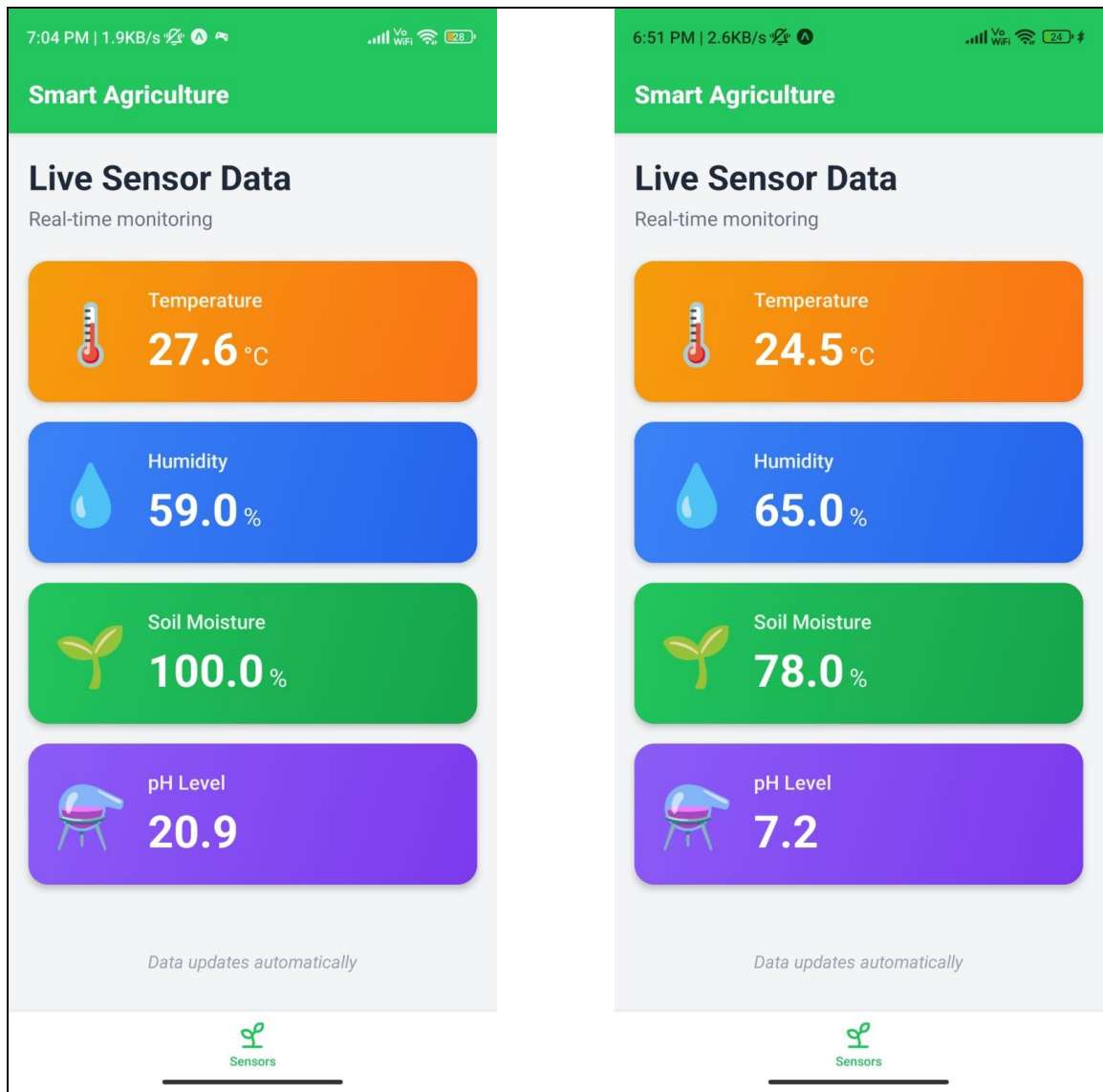
1  import { initializeApp } from 'firebase/app';
2  import { getDatabase } from 'firebase/database';
3
4  const firebaseConfig = {
5    apiKey: process.env.EXPO_PUBLIC_FIREBASE_API_KEY || 'AIzaSyAUxLwgUAlpPcWpp7_5GwHyNHsgNJQ0308',
6    databaseURL: process.env.EXPO_PUBLIC_FIREBASE_DATABASE_URL || 'https://smart-agriculture-df146-default-rtdb.firebaseio.com/',
7  };
8
9  console.log('Firebase Config:', {
10    apiKey: firebaseConfig.apiKey ? 'present' : 'missing',
11    databaseURL: firebaseConfig.databaseURL
12  });
13
14  const app = initializeApp(firebaseConfig);
15  export const database = getDatabase(app);
16  console.log('Firebase initialized successfully');
17

```

Fig:- firebase connectivity

**Observations / Result:**

- The Bolt.AI app displayed live data for Temperature, Humidity, Moisture, and pH.
- Data refreshed every 2 seconds.
- Firebase Realtime Database showed structured JSON data in real time.
- Charts and indicators in the app updated automatically.



Fig;- Bolt.AI App Display

## Troubleshooting & Debugging:

### **1. Firebase Authentication Error:-**

- Problem: The ESP8266 failed to connect to Firebase and displayed an authentication error.
- Cause: Incorrect or missing Firebase authentication token.
- Solution: Retrieved the correct Database Secret Key from the Firebase Console under Project Settings → Service Accounts → Database Secrets and updated it in the Arduino code.

### **2. Data Not Appearing in the App:-**

- Problem: The Bolt.AI app did not display any sensor data even though Firebase showed live updates.
- Cause: Incorrect Firebase Database URL entered in the Bolt.AI configuration.
- Solution: Verified the correct database URL ending with .firebaseio.com and updated it in the app settings.

### **3. Unstable Sensor Readings:-**

- Problem: Moisture and pH readings fluctuated rapidly.
- Cause: Electrical noise and analog signal instability.
- Solution: Added small delays in code and implemented an averaging algorithm to smooth out sensor values.

### **4. Slow UI Response:-**

- Problem: The app UI updated slowly or showed delayed values.
- Cause: Bolt.AI component refresh rate was too low.
- Solution: Enabled real-time data sync and reduced refresh delay in the app configuration.

### **Conclusion:**

In this practical, we successfully developed a mobile application for real-time monitoring of smart agriculture sensor data.

The ESP8266 collected temperature, humidity, soil moisture, and pH data and uploaded it to Firebase in real time.

The Bolt.AI app subscribed to this data and displayed it instantly using text labels and live charts.

Through this implementation, we learned:-

- How to connect IoT sensors to a cloud database and visualize data on a mobile app.
- The importance of real-time synchronization between cloud and app for instant feedback.
- How user interfaces improve usability for monitoring environmental data.

This approach can be extended to full-scale Smart Agriculture systems for remote monitoring and precision farming.